04-1-security-gates

accounts-api security assessment · generated from the repository, not hand-edited

```
# 04-1-security-gates.yaml
#
# Section 4 deliverable: SCA, SAST, full-history secret scanning and IaC
# scanning with the severity thresholds that block a merge.
#
# Assembled by scripts/build-submission.py from the repository.

# ============================================================================
# SOURCE: .github/workflows/security-gates.yml
# ============================================================================
# Section 3 — the scanning tier, made enforceable.
# Threats: T4 (SCA), T2 (SAST), T1/T3 (secrets), T3 (IaC).
#
# THIS WORKFLOW ACTUALLY RUNS on a fresh clone with no secrets and no cloud
# account: every tool is installed from a pinned release in the job, so nothing
# here depends on a licensed action or an org entitlement.
#
# Enforcement contract, in one place so nobody has to read the YAML to know it:
#
#   gate                | day-one mode | blocks on
#   --------------------|--------------|-----------------------------------------
#   waiver-integrity    | BLOCK        | any malformed, unsigned, over-long or EXPIRED waiver
#   secrets (history)   | BLOCK        | any verified finding, any severity, no exceptions
#   iac (conftest)      | BLOCK        | any `deny` rule in policy/rego (data-plane resources)
#   sca (trivy)         | BLOCK        | CRITICAL/HIGH *with a fix available*
#   sast (semgrep)      | WARN -> BLOCK| ERROR severity, p/owasp-top-ten + local rules
#   policy-tests        | BLOCK        | kyverno test / conftest verify failures
#
# Rationale for the SCA line is in docs/03-gates-and-waivers.md. Short version:
# blocking on unfixable CRITICALs produces a queue of waivers nobody can close,
# and a gate that can only be satisfied by waiving is a gate that gets deleted.
name: security-gates

on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "17 2 * * *"        # nightly: full-history secrets + re-scan of shipped digests
  workflow_dispatch:
  workflow_call:                # called in advisory mode by break-glass-deploy.yml

permissions: {}

concurrency:
  group: gates-${{ github.head_ref || github.ref }}
  cancel-in-progress: true

env:
  CONFTEST_VERSION: "0.56.0"
  KYVERNO_VERSION: "1.13.4"
  GITLEAKS_VERSION: "8.21.2"

jobs:
  # ---------------------------------------------------------------------------
  # Runs FIRST and blocks everything else. If the waiver file is broken or a
  # waiver has lapsed, no other gate is allowed to consume the derived ignore
```

```
  # files — because those files are how a lapsed waiver would otherwise keep
  # silently suppressing a finding.
  waiver-integrity:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      id-token: write          # cosign verify-blob (keyless bundle verification)
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with: { persist-credentials: false }
      - uses: sigstore/cosign-installer@dc72c7d5c4d10cd6bcb8cf6e3fd625a9e5e537da # v3.7.0
      - uses: actions/setup-python@0b93645e9fea7318ecaed2b359559ac225c90a2b # v5.3.0
        with: { python-version: "3.12" }
      - run: pip install --disable-pip-version-check pyyaml

      - name: Validate waivers (schema, approver, lifetime, expiry, signature)
        run: python3 scripts/waiver_check.py check --verify-signatures

      - name: Derive per-tool ignore files from ACTIVE waivers only
        run: python3 scripts/waiver_check.py generate

      # The mechanism is only credible if a reviewer can watch it fail. These two
      # fixtures assert that the gate rejects a lapsed waiver and a waiver whose
      # `expires` was edited after approval. If either ever passes, the waiver
      # path has silently become decorative and this job goes red.
      - name: Prove the gate fails closed (negative fixtures)
        run: |
          set -euo pipefail
          if python3 scripts/waiver_check.py check --file waivers/fixtures/invalid-waivers.yaml;
then
            echo "::error::the invalid-waiver fixture PASSED — the waiver gate is not enforcing";
exit 1
          fi
          if python3 scripts/waiver_check.py check --file waivers/fixtures/tampered.yaml --verify-
signatures; then
            echo "::error::the tampered-waiver fixture PASSED — signatures are not being checked";
exit 1
          fi
          echo "waiver gate fails closed on both fixtures, as designed"

      - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
        with:
          name: derived-ignore-files
          path: |
            .trivyignore
            .gitleaks.toml
            .semgrepignore
            policy/rego/data.waivers.json
          retention-days: 7
          include-hidden-files: true

  # ---------------------------------------------------------------------------
  # Secret scanning over FULL GIT HISTORY, not the diff.
  #
  # Mechanism (this is the part that is usually got wrong):
  #   * fetch-depth: 0 — the default shallow clone makes history scanning a no-op
  #     that still exits 0, which is the worst possible outcome.
  #   * `gitleaks detect --log-opts="--all --full-history"` walks every ref,
  #     including deleted branches still reachable from the remote, and every
  #     commit on them. `gitleaks protect` (the default in many templates) only
  #     looks at uncommitted changes.
  #   * On a PR we ALSO run the fast diff-scoped scan so feedback is seconds, but
  #     the full walk runs on every push to main and nightly, so a secret cannot
  #     enter history through a squash-merge or a force-push window.
```

```
  #   * Cost control: the full walk on this repo is ~0.1s; on a 10-year monorepo it
  #     is minutes, which is why it is a push/nightly gate rather than a per-PR one.
  #
  # The binary is installed from a pinned release rather than using
  # gitleaks/gitleaks-action, which requires a paid licence for organisations.
  # A gate that only works on a personal account is not a gate.
  secrets:
    runs-on: ubuntu-24.04
    needs: waiver-integrity
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with:
          fetch-depth: 0                    # REQUIRED. Without this the next step lies.
          persist-credentials: false
      - name: Fetch every remote ref (branches deleted on GitHub still hold blobs)
        run: git fetch --no-tags --prune origin '+refs/heads/*:refs/remotes/origin/*' || true

      - uses: actions/download-artifact@fa0a91b85d4f404e444e00e005971372dc801d16 # v4.1.8
        with: { name: derived-ignore-files }

      - name: install gitleaks
        run: |
          set -euo pipefail
          curl -fsSL -o /tmp/gl.tgz \

"https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}/gitleaks_${GITLEAKS_VE
RSION}_linux_x64.tar.gz"
          tar -xzf /tmp/gl.tgz -C /usr/local/bin gitleaks
          gitleaks version

      - name: gitleaks — full history
        run: |
          set -euo pipefail
          gitleaks detect \
            --source . \
            --config .gitleaks.toml \
            --log-opts="--all --full-history" \
            --redact \
            --report-format sarif --report-path gitleaks.sarif \
            --exit-code 1

      # A finding here is not "delete the commit". History rewriting on a shared
      # repo is a bad first move and does not help: the secret is already on
      # someone's laptop and in GitHub's fork network. The runbook order is
      # ROTATE -> revoke -> then decide about history. See docs/03-gates-and-waivers.md.
      - if: failure()
        run: echo "::error::Secret in history. ROTATE the credential first; rewriting history is
not containment."

      - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
        if: always()
        with: { name: gitleaks-sarif, path: gitleaks.sarif, retention-days: 30 }

  # ---------------------------------------------------------------------------
  sca:
    runs-on: ubuntu-24.04
    needs: waiver-integrity
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with: { persist-credentials: false }
      - uses: actions/download-artifact@fa0a91b85d4f404e444e00e005971372dc801d16 # v4.1.8
        with: { name: derived-ignore-files }
```

```
    # BLOCKING scan: fixable CRITICAL/HIGH only.
    - name: trivy fs — blocking
      uses: aquasecurity/trivy-action@18f2510ee396bbf400402947b394f2dd8c87dbb0 # v0.29.0
      with:
        scan-type: fs
        scan-ref: .
        severity: CRITICAL,HIGH
        ignore-unfixed: true              # see threshold rationale in docs/03
        trivyignores: .trivyignore        # derived from waivers, NOT hand-edited
        exit-code: "1"
        format: sarif
        output: trivy-blocking.sarif

    # NON-blocking scan: everything else, so the unfixable backlog is visible
    # and trendable without being able to stop a merge.
    - name: trivy fs — visibility only
      if: always()
      uses: aquasecurity/trivy-action@18f2510ee396bbf400402947b394f2dd8c87dbb0 # v0.29.0
      with:
        scan-type: fs
        scan-ref: .
        severity: CRITICAL,HIGH,MEDIUM
        ignore-unfixed: false
        exit-code: "0"
        format: table
        output: trivy-informational.txt

    - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
      if: always()
      with: { name: trivy-reports, path: "trivy-*", retention-days: 30 }

# ------------------------------------------------------------------------
sast:
  runs-on: ubuntu-24.04
  needs: waiver-integrity
  permissions: { contents: read }
  steps:
    - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
      with: { persist-credentials: false }
    - uses: actions/setup-python@0b93645e9fea7318ecaed2b359559ac225c90a2b # v5.3.0
      with: { python-version: "3.12" }
    # Version-pinned via pip rather than a container so the job needs no
    # registry pull and no digest that would go stale in this repo.
    - run: pip install --disable-pip-version-check "semgrep==1.99.0"

    # Day one this does not fail the job — WARN. Promotion criterion and the
    # evidence required to promote are in docs/03-gates-and-waivers.md.
    - name: semgrep
      id: semgrep
      run: |
        set -uo pipefail
        semgrep scan --config p/owasp-top-ten --config policy/semgrep/ \
          --severity ERROR --error --sarif -o semgrep.sarif --metrics off
        rc=$?
        echo "rc=$rc" >> "$GITHUB_OUTPUT"
        if [ "$rc" != "0" ]; then
          echo "::warning::SAST findings present. This gate WARNS until 2026-10-15, then BLOCKS."
          {
            echo "### SAST (advisory until 2026-10-15)"
            echo ""
            echo "Semgrep exited \`$rc\`. Findings are in the \`semgrep-sarif\` artefact."
            echo "This gate does not block today; see docs/03-gates-and-waivers.md for the"
            echo "evidence required to promote it."
```

```
            } >> "$GITHUB_STEP_SUMMARY"
          fi
          exit 0

      - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
        if: always()
        with: { name: semgrep-sarif, path: semgrep.sarif, retention-days: 30 }

  # ---------------------------------------------------------------------------
  # IaC. conftest against the HCL directly (no `terraform init`, no registry
  # access, no cloud credentials) so a reviewer can run the identical command
  # offline. Checkov runs alongside for breadth and does NOT block — one tool
  # owns the merge decision, the other supplies coverage. See docs/03.
  iac:
    runs-on: ubuntu-24.04
    needs: waiver-integrity
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with: { persist-credentials: false }

      - name: install conftest
        run: |
          set -euo pipefail
          curl -fsSL -o /tmp/conftest.tgz \
            "https://github.com/open-policy-
agent/conftest/releases/download/v${CONFTEST_VERSION}/conftest_${CONFTEST_VERSION}_Linux_x86_64.tar
.gz"
          tar -xzf /tmp/conftest.tgz -C /usr/local/bin conftest
          conftest --version

      - name: conftest — unit tests for the policies themselves
        run: conftest test --policy policy/rego --parser hcl2 --all-namespaces
policy/fixtures/terraform/good/ && conftest verify --policy policy/rego

      - name: conftest — enforce against terraform (BLOCKING)
        run: conftest test --policy policy/rego --parser hcl2 --all-namespaces terraform/

      # Asserts the rules still reject the production-today configuration. Without
      # this a refactor that accidentally neuters every deny rule would go green.
      - name: conftest — the bad fixture MUST still be rejected
        run: |
          set -euo pipefail
          if conftest test --policy policy/rego --parser hcl2 --all-namespaces
policy/fixtures/terraform/bad/; then
            echo "::error::the known-bad Terraform fixture PASSED — the IaC rules are not
enforcing"; exit 1
          fi
          echo "bad fixture rejected, as designed"

      - name: checkov — breadth, non-blocking
        continue-on-error: true
        uses: bridgecrewio/checkov-action@fcfd4ef72a8469ea29b98a1ba15dc390b1aee1db # v12.2926.0
        with:
          directory: terraform/
          quiet: true
          soft_fail: true
          output_format: cli

  # ---------------------------------------------------------------------------
  # The policies in policy/ are code, and code without tests rots. This job is
  # why a reviewer can trust that the Kyverno rules actually reject the bad
  # manifest rather than merely existing.
  policy-tests:
```

```
    runs-on: ubuntu-24.04
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with: { persist-credentials: false }
      - uses: sigstore/cosign-installer@dc72c7d5c4d10cd6bcb8cf6e3fd625a9e5e537da # v3.7.0
      - uses: actions/setup-python@0b93645e9fea7318ecaed2b359559ac225c90a2b # v5.3.0
        with: { python-version: "3.12" }
      - run: pip install --disable-pip-version-check pyyaml

      - name: install kyverno + conftest
        run: |
          set -euo pipefail
          curl -fsSL -o /tmp/kyverno.tgz \
            "https://github.com/kyverno/kyverno/releases/download/v${KYVERNO_VERSION}/kyverno-
cli_v${KYVERNO_VERSION}_linux_x86_64.tar.gz"
          tar -xzf /tmp/kyverno.tgz -C /usr/local/bin kyverno
          curl -fsSL -o /tmp/conftest.tgz \
            "https://github.com/open-policy-
agent/conftest/releases/download/v${CONFTEST_VERSION}/conftest_${CONFTEST_VERSION}_Linux_x86_64.tar
.gz"
          tar -xzf /tmp/conftest.tgz -C /usr/local/bin conftest

      - run: ./scripts/run-policy-tests.sh

      - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
        if: always()
        with: { name: policy-test-output, path: test-output/, retention-days: 30 }

  # ---------------------------------------------------------------------------
  gate:
    # Single required status check in branch protection. Adding a gate means
    # adding it to `needs:` here — there is one place to look to know what blocks.
    runs-on: ubuntu-24.04
    needs: [waiver-integrity, secrets, sca, sast, iac, policy-tests]
    if: always()
    steps:
      - name: summarise
        run: |
          set -uo pipefail
          fail=0
          {
            echo "### Security gates"
            echo ""
            echo "| gate | result | blocking |"
            echo "|---|---|---|"
          } >> "$GITHUB_STEP_SUMMARY"
          add() {
            printf '| %s | %s | %s |\n' "$1" "$2" "$3" >> "$GITHUB_STEP_SUMMARY"
            if [ "$3" = "yes" ] && [ "$2" != "success" ]; then fail=1; fi
          }
          add waiver-integrity "${{ needs.waiver-integrity.result }}" yes
          add secrets          "${{ needs.secrets.result }}"          yes
          add sca              "${{ needs.sca.result }}"              yes
          add iac              "${{ needs.iac.result }}"              yes
          add policy-tests     "${{ needs.policy-tests.result }}"     yes
          add sast             "${{ needs.sast.result }}"             "no (until 2026-10-15)"
          exit $fail
```